Assignment: Crop Type Classification

1. About Solafune and the Agriculture Team

Solafune develops geospatial analytics products and services that transform satellite and other Earth-observation data into practical insights. In agriculture, satellite imagery can support crop monitoring, crop-type mapping, seasonal analysis, and the identification of agricultural activity across large areas.

Crop classification is one example of how satellite imagery can be used to derive useful agricultural information. Different crops have different spectral characteristics and seasonal growth patterns. By analysing multi-temporal Sentinel-2 imagery, it is possible to train models that identify crop classes at pixel or parcel level.

This assignment reflects a simplified version of an agricultural remote-sensing workflow: using multi-temporal Sentinel-2 imagery and labelled crop data to build and evaluate a crop classification model.

2. Assignment Context

You will be provided with a subset of satellite-image patches and their corresponding crop-type labels. Your task is to explore the data, develop a classification approach, train a model, evaluate the outputs, and communicate your reasoning and findings clearly.

The purpose of this exercise is not to achieve the highest possible classification accuracy. We are more interested in:

- Your reasoning when approaching a geospatial machine-learning problem
- Your ability to understand, inspect, and prepare geospatial raster data
- The clarity and reproducibility of your code
- Your engineering practices and project structure
- Your interpretation of model outputs and limitations
- Your ability to communicate technical work clearly

3. Dataset Description

The assignment dataset is derived from the PASTIS dataset https://github.com/VSainteuf/pastis-benchmark/tree/main, a benchmark dataset for crop-type classification and semantic segmentation using multi-temporal Sentinel-2 imagery.

The dataset is organised into image patches. Each patch represents the same geographic area observed repeatedly over time by the Sentinel-2 satellite constellation.

For this assignment, candidates will receive Sentinel-2 image files and their corresponding annotation files.

```
None
DATA_S2/
    S2_<patch_id>.npy

ANNOTATIONS/
    TARGET_<patch_id>.npy
```

For example:

➔ DATA_S2/S2_30003.npy
➔ ANNOTATIONS/TARGET_30003.npy

Sentinel-2 Data:

Each Sentinel-2 file is stored as a NumPy array with the following structure:

```
None
(number_of_observations, number_of_bands, height, width)
```

The 10 Sentinel-2 bands are typically provided in the following order:

Band Index	Sentinel-2 Band	Common Name
0	B2	Blue
1	B3	Green
2	B4	Red
3	B5	Red Edge 1
4	B6	Red Edge 2
5	B7	Red Edge 3
6	B8	Near Infrared
7	B8A	Narrow Near Infrared
8	B11	Short-Wave Infrared 1
9	B12	Short-Wave Infrared 2

Annotation Data:

Each annotation file is stored as a NumPy array containing semantic segmentation labels.

The annotation array has the following structure:

```
None
(number_of_annotation_layers, height, width)
```

For this assignment, we are giving only the **0th layer** of the annotation array:

```
None
(1,height, width)
```

Each integer pixel value represents a semantic class. The class IDs correspond to crop types or other land-cover categories defined in the PASTIS label scheme.

Label and Color	Class Name
0	*Background*
1	Meadow
2	Soft winter wheat
3	Corn
4	Winter barley
5	Winter rapeseed
6	Spring barley
7	Sunflower
8	Grapevine
9	Beet
10	Winter triticale
11	Winter durum wheat
12	Fruits, vegetables, flowers
13	Potatoes
14	Leguminous fodder
15	Soybeans
16	Orchard
17	Mixed cereal
18	Sorghum
19	*Void label*

Temporal Metadata

The original PASTIS dataset also includes temporal metadata describing the acquisition dates of Sentinel-2 observations. This is also provided along with the sentinel and annotation data named ***metadata.geojson***

The objective is to build a clear and reproducible crop-classification workflow using the provided Sentinel-2 and annotation data. Candidates are encouraged to make reasonable assumptions where needed, document those assumptions, and explain the implications for their modelling approach.

4. Tasks

A. Data Loading and Preparation

Load the Sentinel-2 and target NumPy files and create a clear data-loading workflow.

Your implementation should include:

- Reading Sentinel-2 `.npy` files
- Reading target `.npy` files
- Inspecting array dimensions and data types
- Data preprocessing
- Preparing the data in a suitable format for model training

You may use a subset of the data during development if needed. Please explain the sampling strategy and why it was selected.

B. Exploratory Data Analysis

Perform exploratory analysis to understand the dataset before training a model.

Your exploration should include but not limited to:

- Number of available image patches
- Image dimensions and Sentinel-2 band structure
- Number of temporal observations per patch
- Available crop or land-cover classes
- Class distribution across the dataset
- Visualisation of selected label patches
- Visualisation of Sentinel-2 imagery using an appropriate RGB-band combination, if possible
- AOI extent or approximate geographic coverage
- Any useful information you can infer about the agricultural landscape

Please include clear plots, maps, or figures where useful.

C. Train / Validation / Test Split

Create a train, validation, and test split.

Please explain your split strategy. Save the patch IDs assigned to each split in separate text files so that the split can be reproduced.

D. Model Training

Train a crop classification model using the provided data.

You may choose either:

- A traditional machine-learning model, such as Random Forest, XGBoost, LightGBM, Support Vector Machine, or another suitable method
- A deep-learning model, such as U-Net, DeepLabV3+, CNN-based segmentation models, transformer-based models, or another suitable architecture

The model choice is up to you. Please select an approach that is appropriate for your available compute resources.

If you prefer to train a deep-learning model but do not have local GPU access, you may use Google Colab or another cloud-based environment with GPU support.

A simpler baseline model with strong reasoning, clean implementation, and thoughtful interpretation is preferable to an unnecessarily complex model with limited explanation.

Please document:

- Input features used by the model
- Any preprocessing, normalization, or feature engineering
- Hyperparameters and training configuration
- How you handled class imbalance, if applicable
- Compute environment or hardware constraints, if relevant

Please save the final trained model, model checkpoint, or model weights where practical.

If the saved model file is too large to include in the GitHub repository, please do not upload it directly. Instead, include clear instructions in the README explaining how the model can be reproduced, and optionally provide a download location or Git LFS configuration if applicable.

E. Model Evaluation

Evaluate the trained model using the validation and test datasets.

Suggested evaluation outputs include:

- Overall accuracy
- Per-class precision, recall, and F1-score, mIoU, etc.
- Confusion matrix
- Any other relevant metrics or observations

F. Visualisation of Outputs

Visualise the model outputs for selected test patches.

You can show:

- Ground-truth label | Model prediction
- Sentinel-2 RGB image | Ground-truth label | Model prediction
- Any other relevant information

G. Analysis and Interpretation

Include a short analysis section discussing your approach and results.

This section should cover:

- Why you chose the selected model
- Strengths and limitations of the approach
- Interpretation of the evaluation results
- Classes that performed well or poorly
- Possible reasons for confusion between specific crop classes
- Effects of class imbalance, limited samples, or label quality
- Effects of spatial resolution, cloud contamination, missing observations, or seasonal timing
- Any useful information or assumptions about the AOI
- Recommended next steps for improving the model or dataset

You do not need to conduct external research about the AOI. However, if you find relevant information that helps interpret the agricultural landscape, crop calendar, climate, or crop types, you may include it with appropriate references.

Write a short report and Create: **report.md**

5. Expected Deliverables

Please submit a GitHub repository containing the following:

```
None
project-root/
├── README.md
├── report.md
├── requirements.txt
├── notebooks/
│   └── exploration_and_training.ipynb
├── src/
│   ├── data_loading.py
│   ├── preprocessing.py
│   ├── train.py
│   ├── evaluate.py
│   └── visualization.py
├── outputs/
│   ├── figures/
│   ├── metrics/
│   └── predictions/
└── configs/
    └── config.yaml
```

The exact repository structure is flexible. You may organise the project differently if your structure is clear and reproducible.

Your repository should include:

- **Code** Python scripts and/or notebooks used for:
  - Data loading
  - Data exploration
  - Preprocessing
  - Model training
  - Validation and testing
  - Visualisation
  - Saving outputs and metrics
- **README.md**
  - Project overview
  - Dataset structure
  - Environment setup
  - Required dependencies
  - How to run the code
  - How to reproduce the main results
  - Your overall approach

    - Key assumptions
    - Any limitations or known issues
- **report.md**
- **Additional Files** You may include any additional files that support reproducibility and clarity, such as:
    - `requirements.txt` or `environment.yml`
    - Configuration files
    - Saved trained model weights
    - Utility scripts
    - Experiment logs
    - Figures and metric outputs

Note : Please do not upload large files to the repository unless explicitly requested. Instead, describe where the data should be placed locally and how the expected folder structure should look. Optionally provide a download location or Git LFS configuration if applicable.